In-Memory Object Storage in a Single Address Space

Taj Jethwani-Keyser

May 4, 2026

Harvard SEAS

Outline

Motivation

Approach

Benchmarks

Motivation

What is an in-memory object store?

- A shared key→value table holding live application objects.
- Multiple clients (threads or processes) read and write concurrently.
- Latency profile ranging from nanoseconds to milliseconds.
- Used as a cache, a feature store, or a coordination layer between tightly coupled services.
  - Often accessed remotely, but not always (e.g., data pipelines, multi-GPU training).
  - Focus today is on low-latency intra-server IPC.

In-Memory Bottlenecks

Can classify (most) overhead into one of two categories:

IPC overhead

- Context-switch on every `get`/`put`:
  - Notify database with `send`/`recv`
  - Schedule database worker thread
- Copying values from client memory to server memory and back

Metadata overhead

- Synchronization on shared state
  - Concurrent hash tables, skip lists
  - Key/value reference counts
- Memory access isn't free!
  - Data cache and TLB misses

Related Work

- "Rearchitecting In-Memory Object Stores for Low Latency" (VLDB '22).
- Elegant solution to the IPC problem:
  - Metadata (hash table) and data (raw buffers) allocated from a shared memory region.
  - Clients read/write to database directly via specialized client library.
- Unsurprisingly, benchmarks are far better than competitors like Redis and Plasma.[1]
- But... focus is placed on safety rather than "low latency":
  - "To avoid concurrent modification to the object store's state, we use a single spinlock to serialize client operations... Lightning has the best performance when there is only one client because multiple clients can result in lock contention."
  - Emphasis on fault isolation (formally verified!).

[1] deprecated Apache Arrow project

Towards a Single Address Space

- Suppose that clients never crash.
- Give up on the multi-process model:
  - Shared memory is often slower (due to TLB pressure) and harder to deal with (must use relative offsets instead of raw pointers) than local heap memory.
  - Ideally, a client can `put` an arbitrary pointer (e.g., to a graph or sparse matrix) and it is immediately accessible to other clients, like in multi-threaded applications.
  - Lose process isolation, but this is (partially) fixable.
- Use faster, harder-to-verify data structures like concurrent hash tables.
- But maybe it's impractical to refactor your codebase to use threads!

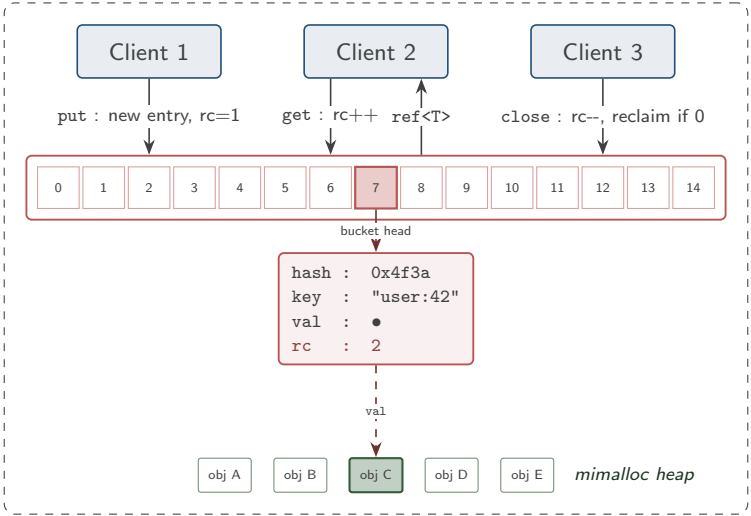
These motivate a design where *client binaries are loaded into a single process at runtime*.

Approach

System API

- Clients `include` a header with database operations, resolved at runtime to host code:
  - `put(key, value*, dtor)`: insert pointer to value and register destructor callback.
  - `get(key) -> handle`: retrieve ref-counted pointer to value.
  - `close(handle)`: decrement ref-count, call `dtor` and free when it reaches zero.
- Also some convenience functions, like a C++-only RAII helper for handles.
- Reference counts make the API flexible but are a real bottleneck at scale.

System Architecture



Concurrent Hash Tables

- For the database backend, prioritize throughput and tail latency at high concurrency.
- Initially tried `boost::concurrent_flat_map`:
  - Flat open-addressing scheme that uses SIMD for hash lookups.
  - Striped read-write locks and stop-the-world resizes.
- Optimization: store atomic pointers and use CAS for writes.
  - Allows updates while holding a read lock instead of a write lock.
  - But requires more advanced reclamation strategies (see next slide).
- Even better: fully lock-free hash table with chained buckets and cooperative resizes.

Safe Memory Reclamation

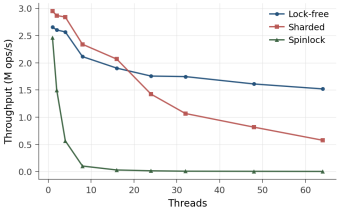
- Approaches #2 and #3 require synchronization to ensure that a value is not freed between retrieving the pointer and incrementing the ref-count.
- Traditional solution is *epoch-based reclamation*.
  - Threads announce a current "epoch"; values deleted in epoch t are not freed until all threads reach epoch $t + 1$.
  - Single memory barrier per read, but time-to-free is unbounded.
- Alternative solution: *hazard pointers*.
  - Threads announce the specific pointers they are reading (i.e., the current hash table and value); when freeing a batch of values, must check other threads' hazard lists first.
  - Much heavier and harder to implement, but stalled threads block at most two frees.

Benchmarks

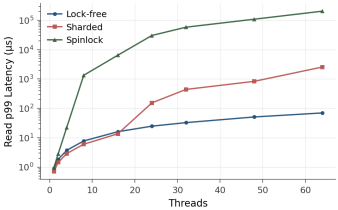
Experimental Setup

- Workload: Zipfian ($\theta = 0.99$) over 4M keys, 50% read/write, 16 threads.
  - Sweep each of these parameters, fix the others.
  - Workers pinned to a specific core (except TLB benchmark!).
  - Only microbenchmarks for now; will compare with Lightning on YCSB.
- Measurement: 5 s timed window, 2 s warmup, sampled per-op `rdtsc` (1-of-128 ops).
- Hardware: AMD EPYC 7452 (32 cores @ 2.35 GHz), 125 GB memory, 128 MB L3.

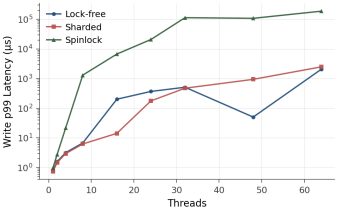
Hash Tables: Concurrency



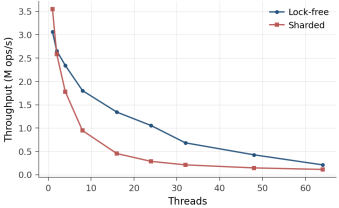
Throughput vs. threads



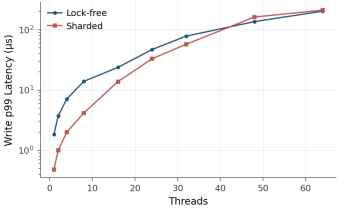
Read p99 latency vs. threads



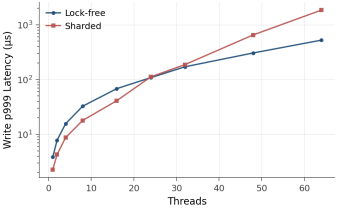
Write p99 latency vs. threads



Fill throughput vs. threads

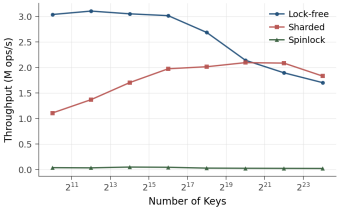


Fill p99 latency vs. threads

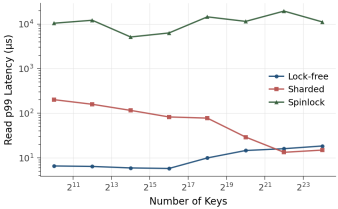


Fill p999 latency vs. threads

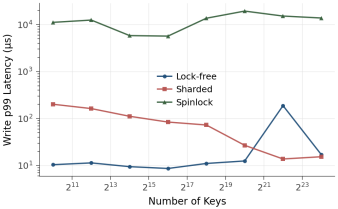
Hash Tables: Working Set



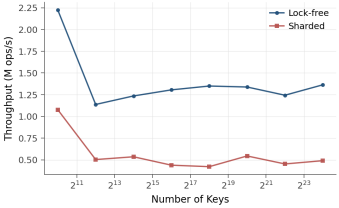
Throughput vs. working set



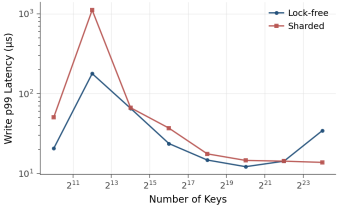
Read p99 latency vs. working set



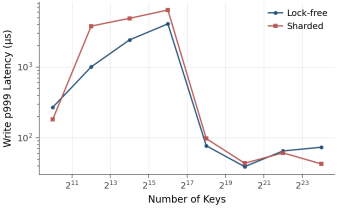
Write p99 latency vs. working set



Fill throughput vs. working set

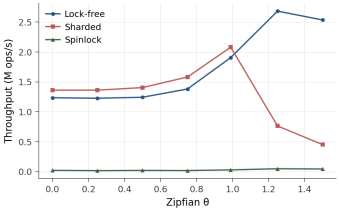


Fill p99 latency vs. working set

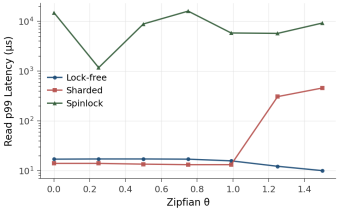


Fill p999 latency vs. working set

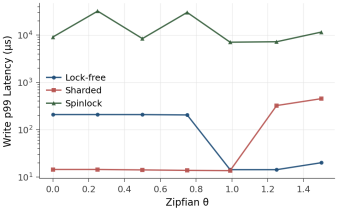
Hash Tables: Zipfian θ



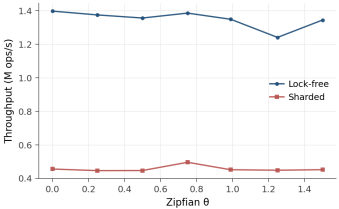
Throughput vs. θ



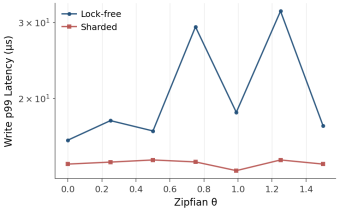
Read p99 latency vs. θ



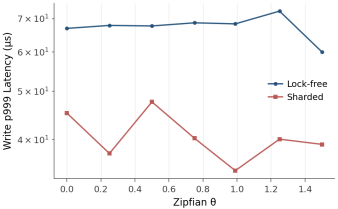
Write p99 latency vs. θ



Fill throughput vs. θ

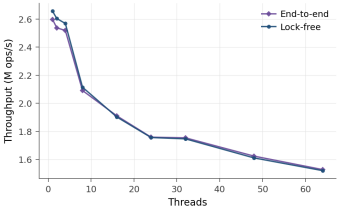


Fill p99 latency vs. θ

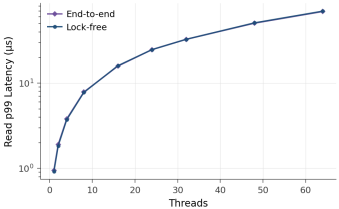


Fill p999 latency vs. θ

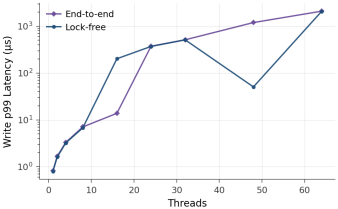
Client Library Overhead



Throughput vs. threads

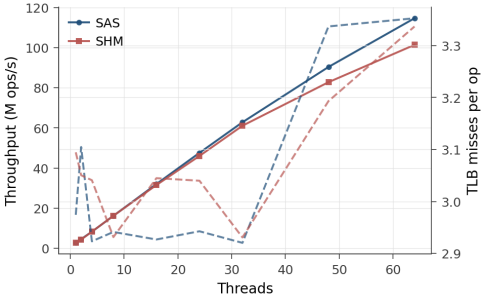


Read p99 latency vs. threads

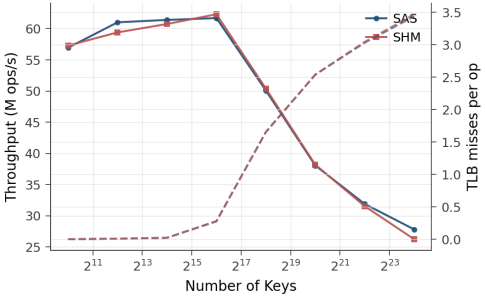


Write p99 latency vs. threads

Shared Memory Overhead



Throughput + TLB misses vs. threads



Throughput + TLB misses vs. working set

Questions?